ALPHA-MAXIMIZED STRATEGY REPORT

Date: 2026-01-16

1. EXECUTIVE SUMMARY

The Alpha-Maximized Strategy represents the 'Bedrock' of the portfolio. It is a machine learning-based approach using Gradient Boosting Regressors (GBM) to capture non-linear relationships between macro factors and cyclical/credit asset returns.

PERFORMANCE VERIFICATION (Multi-Timeframe):

- Overall robustness winner (Score 43 vs 39 for ERP Regime)
- Statistically significant in 3/5 timeframes tested
- Sharpe Ratio (Full History): 1.58
- Sharpe Ratio (1-Year): 2.48
- Volatility Target: 20%

CORE PHILOSOPHY:

Uses macro regimes (Rates, Dollar, Credit Spreads) to predict momentum in cyclicals (Energy, Materials, Industrials) and safe havens (Gold).

2. STRATEGY LOGIC & CONFIGURATION

UNIVERSE (Target Assets):
  * XLB (Materials)
  * XLI (Industrials)
  * XLE (Energy)
  * JNK (High Yield Bond)
  * GLD (Gold)

MACRO INPUTS (Feature Drivers):
  * ^TNX (10-Year Treasury Yield): Rate change and trend
  * UUP (US Dollar Index): Global risk interactions
  * JNK/IEF Ratio: Proxy for credit spreads/liquidity

MODEL ARCHITECTURE:
  * Algorithm: Gradient Boosting Regressor (sklearn)
  * Parameters: n_estimators=50, max_depth=3, learning_rate=0.05
  * Training: Rolling window or expanding window (min 100 days)

RISK MANAGEMENT:
  * Volatility Target: 20% annualized
  * Position Sizing: Long-only, Proportional to predicted positive return
  * Leverage Cap: 2.0x

3. PYTHON IMPLEMENTATION (PART 1)

```
import numpy as np
import pandas as pd
from sklearn.ensemble import GradientBoostingRegressor

# CONFIGURATION
TARGET_ASSETS = ['XLB', 'XLI', 'XLE', 'JNK', 'GLD']
MACRO_ASSETS = ['^TNX', 'UUP', 'IEF', 'SHY', 'JNK']
TARGET_VOL = 0.20

class AlphaMaxStrategy:
    def __init__(self):
        self.models = {}
        self.is_trained = False

    def _create_features(self, prices, macro_data):
        df = pd.DataFrame(index=prices.index)
        # 1. Macro Features
        if '^TNX' in macro_data.columns:
            tnx = macro_data['^TNX']
            df['rate_change'] = tnx.diff(20)
            df['rate_trend'] = tnx - tnx.rolling(60).mean()
        if 'JNK' in macro_data.columns and 'IEF' in macro_data.columns:
            df['credit_spread'] = macro_data['JNK'] / macro_data['IEF']
        if 'UUP' in macro_data.columns:
            df['dollar_mom'] = macro_data['UUP'].pct_change().rolling(20).mean()

        # 2. Asset Features
        targets = prices.pct_change().shift(-1)
        asset_map = {}
        for ticker in TARGET_ASSETS:
            if ticker not in prices.columns: continue
            col_list = []
            r = prices[ticker].pct_change()
```

3. PYTHON IMPLEMENTATION (PART 2)

```
    def train(self, prices, macro_data):
        features, targets, asset_map = self._create_features(prices, macro_data)
        common_idx = features.dropna().index.intersection(targets.dropna().index)
        if len(common_idx) < 100: return

        for ticker, cols in asset_map.items():
            X = features.loc[common_idx][cols]
            y = targets.loc[common_idx][ticker]
            model = GradientBoostingRegressor(n_estimators=50, max_depth=3,
                                              learning_rate=0.05, random_state=42
            model.fit(X, y)
            self.models[ticker] = model
        self.is_trained = True

    def generate_signals(self, prices, macro_data):
        if not self.is_trained: self.train(prices, macro_data)
        features, _, asset_map = self._create_features(prices, macro_data)
        signals = {}

        for ticker, cols in asset_map.items():
            row = features.iloc[-1:][cols]
            if not row.isnull().values.any():
                signals[ticker] = self.models[ticker].predict(row)[0]

        # Position Sizing (Long Only)
        raw_w = pd.Series(signals)
        raw_w[raw_w < 0] = 0
        if raw_w.sum() == 0: return pd.Series(0, index=raw_w.index)
        weights = raw_w / raw_w.sum()

        # Vol Targeting
        recent_vols = prices.pct_change().iloc[-20:].std() * np.sqrt(252)
        port_vol = (weights * recent_vols[weights.index]).sum()
```